

ABB AC450 Migration Studio Guide

From ABB AC450 source truth to Valmet-ready engineering deliverables

Version 1.0.0



What Is ABB AC450 Migration Studio?

Platform at a Glance

System Architecture

User Workflow: From File to Final Deliverable

Live Links & Developer Access

Technology Stack & Development Tools

Developer Note

Contact Details

WHAT IS ABB AC450 MIGRATION STUDIO?

ABB AC450 Migration Studio is an engineering conversion platform designed to transform ABB Advant Controller 450 engineering source data into structured, reviewable and Valmet-ready engineering deliverables.

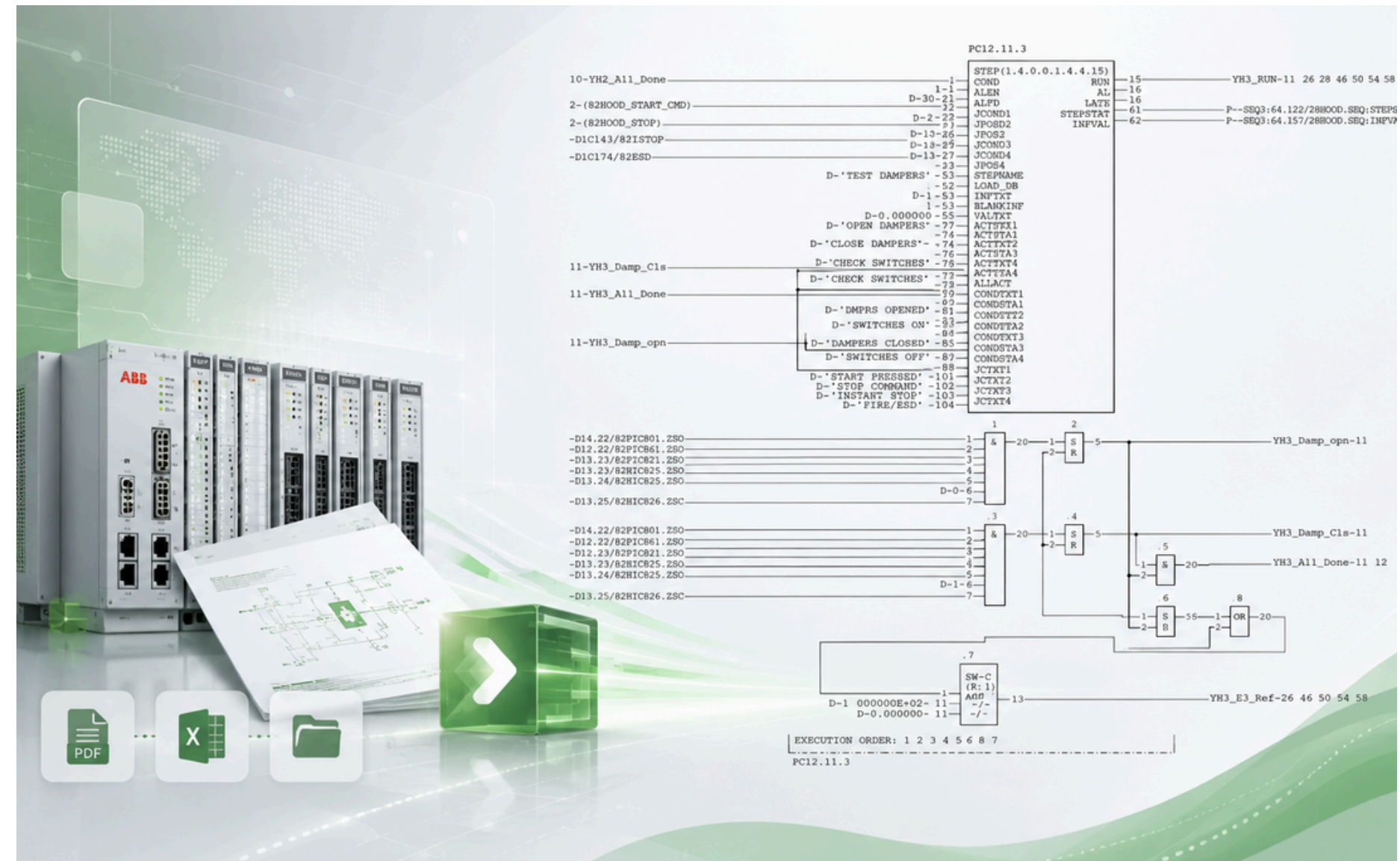
The platform automates repetitive engineering activities including:

Extraction → Validation → Clubbing → Comparison → Address Arrangement → Template Generation

It supports ABB engineering data originating from PDF, AAX, BAX, and Excel sources.

CORE PRINCIPAL

The software accelerates engineering work; the engineer retains responsibility for validation and acceptance.

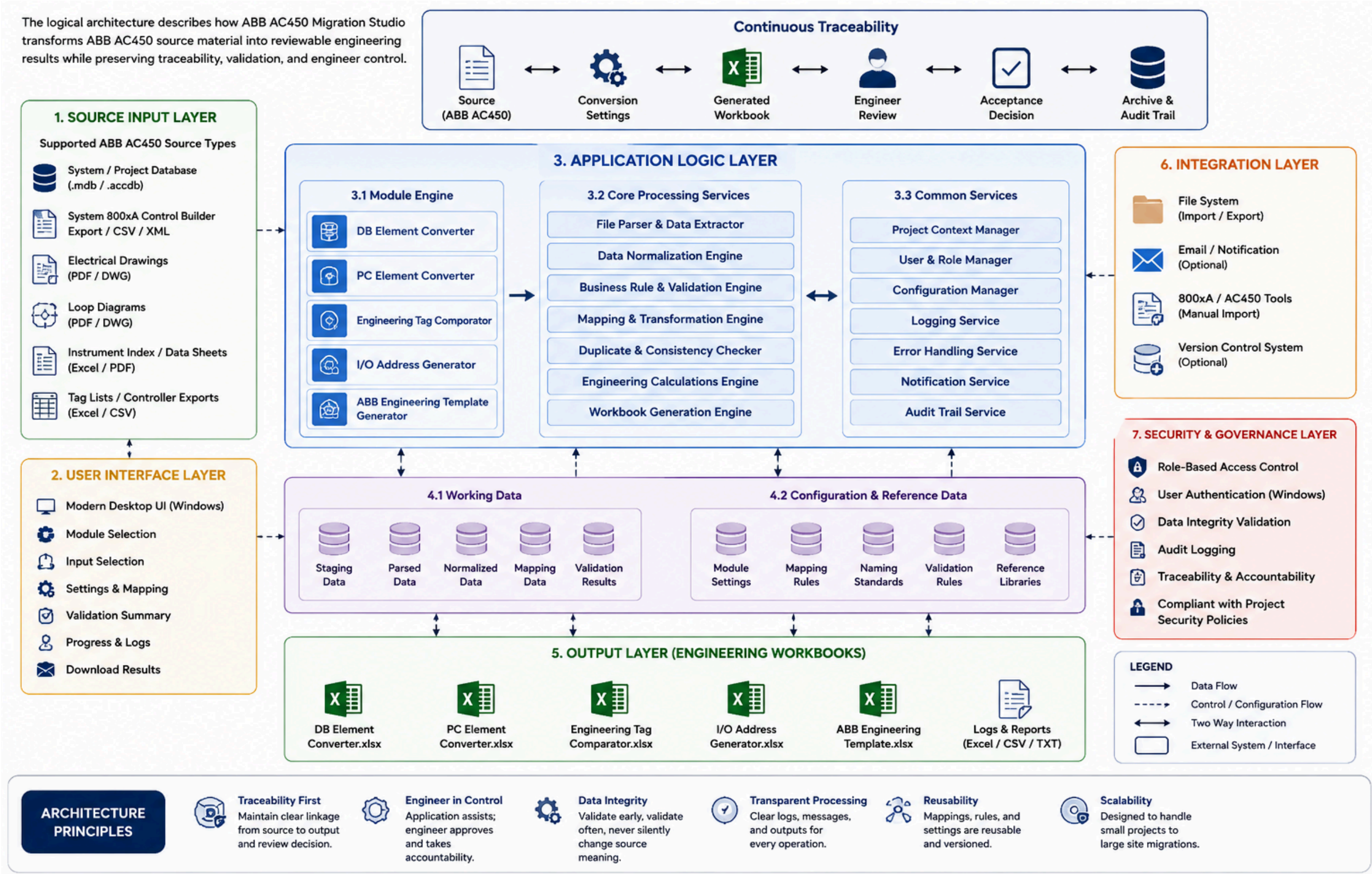


PLATFORM AT A GLANCE

Sr. No.	Module	Purpose	Input
1	DB Element Converter	Extract and structure DB engineering information	PDF / BAX
2	PC Element Converter	Extract PC I/O and logic references	PDF / AAX
3	Engineering Tag Comparator	Compare engineering tag populations	Excel / Excel
4	I/O Address Generator	Allocate engineering records to card/channel addresses	Generated Excel
5	ABB Engineering Template Generator	Produce structured engineering template output	Processed Excel

Sr. No.	Family	Purpose
1	Analog Input	AI AI800_
2	Analog Output	AO AO800_
3	Digital Input	DI DI800_
4	Digital Output	DO DO800_

SYSTEM ARCHITECTURE



The ABB AC450 Migration Studio uses a layered architecture where the Next.js frontend provides the Valmet-branded user interface, FastAPI exposes the REST APIs, and dedicated engineering engines process DB, PC, comparison, I/O arrangement, and template workflows. PDF, AAX, BAX, and Excel data are validated, transformed, clubbed, and converted through backend services before standardized Excel deliverables are generated and returned to the user through the Vercel-Render production environment.

USER WORKFLOW: FROM FILE TO FINAL DELIVERABLE

RECOMMENDED OPERATING SEQUENCE

Step 1 : Prepare and preserve the original ABB source files.

Step 2 : Run the appropriate converter: DB Element Converter or PC Element Converter

Step 3 : Review the generated engineering dataset.

Step 4 : Run the Engineering Tag Comparator when reconciliation is required.

Step 5 : Run the I/O Address Generator on the accepted dataset.

Step 6 : Run the ABB Engineering Template Generator.

Step 7 : Perform engineering review and acceptance.

Step 8 : Archive source, outputs, and review records.

GOLDEN WORKFLOW

Source → Conversion → Verification → Arrangement → Template → Review → Issue

LIVE LINKS & DEVELOPER ACCESS

Resource	Link
Production Website	https://abb-ac450-migration-studio.vercel.app/
Frontend	https://abb-ac450-migration-studio.vercel.app/
Backend API	https://valmet-abb-ac450-api.onrender.com/
Health Check	https://valmet-abb-ac450-api.onrender.com/health
API Documentation	https://valmet-abb-ac450-api.onrender.com/docs
GitHub Repository	https://github.com/valmet-intern-official/ABB-AC450-Migration-Studio
Associated Account	valmet.intern@gmail.com

TECHNOLOGY STACK & DEVELOPMENT TOOLS

BACKEND TECHNOLOGIES

Technology / Tool	Purpose
Python 3.11	Core backend runtime used for engineering data processing and conversion services
FastAPI	REST API framework used to expose upload, processing, status, download, and logging services
Uvicorn	ASGI server used to run the FastAPI backend in development and production
Pydantic v2	Request, response, and engineering-data validation models
pydantic-settings	Centralized backend configuration and environment-variable management
python-multipart	Handles multipart file uploads from the web application
Thread Pool Executor	Runs long-running conversion jobs without blocking the FastAPI event loop
Job Manager	Tracks upload, processing, progress, completion, failure, and generated-output state

FRONTEND TECHNOLOGIES

Technology / Tool	Purpose
Next.js 15	Main frontend framework and application runtime
React 18	Component-based UI development
TypeScript	Type-safe frontend development and API/data contracts
Tailwind CSS 3	Styling and responsive Valmet-branded interface
Zustand	Frontend state management for modules, uploads, jobs, and processing state
Axios	Communication between frontend and backend APIs
Framer Motion	UI animations and transition effects
Lucide React	Engineering and application interface icons
clsx	Conditional CSS class handling

TECHNOLOGY STACK & DEVELOPMENT TOOLS

API & COMMUNICATION

Technology / Tool	Purpose
REST API	Communication interface between frontend and backend
FastAPI API Layer	Provides application endpoints for all five engineering modules
Axios	Sends upload, process, status, log, and download requests
JSON	Standard request and response format for processing metadata and job status
Multipart/Form-Data	Transfers PDF, AAX, BAX, and Excel files to the backend
OpenAPI / Swagger	Interactive API documentation generated by FastAPI

ENGINEERING & FILE PROCESSING

Technology / Tool	Purpose
pdfplumber	Layout-aware extraction of engineering information from PDF DB documents
PyMuPDF (fitz)	Fast PDF text and page processing, especially for PC diagrams
AAX Parser	Reads ABB AC450 PC Element AAX exports and extracts engineering/I/O information
BAX Parser	Reads ABB AC450 DB Element BAX exports
XlsxWriter	High-volume Excel workbook generation where required
pandas	Tabular data manipulation and transformation
Pillow	Image processing support for optional OCR workflows
Tesseract OCR	Optional OCR processing for difficult PDF content
Custom Parsing / Grammar Engine	Detects engineering patterns, categories, Device Tags, Loop Tags, addresses, and related structures

TECHNOLOGY STACK & DEVELOPMENT TOOLS

PROCESSING ENGINES

Technology / Tool	Purpose
DB Element Engine	Extracts, validates, maps, clubs, and exports DB engineering information
PC Element Engine	Extracts I/O and control references from PC PDFs and AAX files
Engineering Tag Comparator	Compares Device Tag populations across two Excel workbooks
I/O Address Generator	Allocates engineering records into defined card/channel structures
ABB Engineering Template	Maps processed engineering records into ABB/Valmet template fields
Clubbing Engine	Groups related engineering records using Loop Tag relationships
Category Mapping Engine	Converts categories into AI/AO/DI/DO/800-series indicator columns
Validation Engine	Detects invalid, incomplete, duplicate, or inconsistent engineering records
Function Block Analyzer	Detects and counts functional blocks such as PIDCON, MOTCON, VALVECON, and MANSTN

DATABASE / STORAGE

Technology / Tool	Purpose
File-based Job Storage	Maintains processing-job state without requiring an application database
Temporary File System	Stores uploaded files and generated Excel outputs during processing
JSON Job Metadata	Persists job status and processing information
Excel Workbooks	Primary engineering input/output data format

TECHNOLOGY STACK & DEVELOPMENT TOOLS

COAD STORAGE & VERSION CONTROL

Technology / Tool	Purpose
Git	Local source-code version control
GitHub	Central project repository and source-of-truth code storage
GitHub Main Branch	Production deployment branch
GitHub Releases	Versioned application releases and deployment history

DEPLOYMENT & HOSTING

Technology / Tool	Purpose
Vercel	Hosts and serves the Next.js production frontend
Render	Hosts the FastAPI backend and engineering processing engines
GitHub Auto Deploy	Source-triggered deployment workflow
Environment Variables	Stores production API and runtime configuration
HTTPS	Secure communication between browser and production backend

DEVELOPER NOTE

THIS IS A COMPILER, NOT JUST AN EXCEL CONVERTER.

You are not maintaining an Excel converter. You are maintaining a compiler with an Excel backend.

The source files are the truth. The parser's job is to recover engineering facts, the processing layer's job is to apply deterministic rules, and the Excel layer's job is only to represent the result. Do not move business logic downstream just because it is easier there. A regex belongs in the reader. A clubbing rule belongs in the domain layer. A column-order change belongs in the export contract. And a missing hardware address is a data condition—not an invitation to invent one.

When you touch the AAX/BAX engines, assume that somewhere in a 10-line fragment an engineer has hidden a dependency on one token, one suffix, one :ADDR, or one channel number. Preserve the full token before you normalize it. MV, RUN, STOP, FAULT, SV1, SV2, GSO, GSC are not cosmetic strings. They can determine what downstream logic thinks the record is. The same applies to AI800_ versus AI: the underscore is part of the grammar, not decoration.



CONTACT DETAILS

DEVELOPER / MAINTAINER

Vedant Lanjekar

DEVELOPER EMAIL

vedantlanjekar456@gmail.com

VALMET INTERNSHIP / PROJECT CONTACT

valmet.intern@gmail.com

VALMET WEBSITE

www.valmet.com

VALMET OFFICE

Valmet Technologies and Services Pvt. Ltd.
Office No. 302, Global Port, Mumbai Bangalore Highway,
Baner, Pune – 411 045, Maharashtra, India

